

MoonBit 并行 FFI 库的规范性规格

第一部分定义 Plan、所有权与分离安全调度的工程合同

朱哲皓

Luna-Flow

GitHub: KCN-judu

摘要

本文把一个 MoonBit 并行 FFI 库规定为关于 Plan 表示、所有权转移和并发 buffer 安全的合同。语义核心不是重新定义 `map` 与 `reduce` 的基础函数意义，而是精确回答三个跨边界致命问题：MoonBit 如何构造纯并行 Plan，该 Plan 如何以 ABI 受约束的 payload 形式跨越 FFI，以及共享 C runtime 如何在不产生数据竞争和生命周期错误的前提下执行 chunk 化的 fork-join 工作。第一部分定义 Plan ADT、布局同构约束、线性所有权状态机、borrow / transfer 规则，以及 chunk 调度的分离检查。第二部分给出 native 与 JavaScript 目标的实现化与符合性义务。第三部分把这些义务细化为实现层面的一致性检查，覆盖异常控制流、字节级布局校验、安全算子准入与 fail-closed 验证场景。英文文本是权威版本，中文文本是严格镜像。

关键词：MoonBit, FFI, OpenMP, separation safety, linear ownership, ABI, engineering specification

第一部分：语义核心

规范性范围

本规格把一个 MoonBit 并行库的 version-1 合同划分为三层：

- 构造纯并行 Plan 的 MoonBit facade；
- 将 Plan 下放为 ABI 受约束 payload 的后端特化 FFI 边界；
- 将 payload 解释为 chunk 化 fork-join 执行的共享 C runtime。

具有规范约束力的内容仅限于：

- MoonBit 侧 Plan 的代数形状；
- 允许穿越 FFI 的值类别；
- zero-copy 或借用视图所要求的布局同构条件；
- MoonBit 堆和 C 可见堆之间的线性所有权状态转移；
- race-free chunk 调度所要求的分离逻辑义务；
- native 与 JavaScript 目标的 faithful realization 所必须满足的符合性义务。

调度启发式、OpenMP 调优细节和局部优化策略均不属于规范性内容，除非它们影响外部可观察的所有权、布局或竞争安全合同。

记号与元层约定

本文使用如下记号:

- H_M 表示 MoonBit 堆, H_C 表示 C / 宿主可见堆;
- Γ 表示跨 FFI 边界携带的所有权资源上下文;
- b 表示 buffer 身份, v 表示 view, p 表示 plan, d 表示 runtime descriptor;
- mapsto 表示元层部分求值;
- $\text{Own}_M(b)$ 、 $\text{Borrow}_C(b)$ 、 $\text{Own}_C(b)$ 表示所有权状态;
- $\Gamma \rightarrow \Gamma'$ 表示一步所有权转移;
- $P * Q$ 表示不相交堆片段上的分离合取;
- $\text{Interval}(i, j)$ 表示半开区间 $[i, j)$;
- $\text{Chunks}(n, m)$ 表示把 $[0, n)$ 划分为 m 个子区间的分块;
- $\text{Den}(p)$ 表示 plan 指称;
- $t \dashv p \mapsto q$ 表示在目标 t 上将 MoonBit plan p 下放为 payload q ;
- $\text{parse}(t, q) \mapsto d$ 表示包装层将 payload 解析为 runtime descriptor d ;
- $\text{exec}(d) = y$ 表示 runtime descriptor d 执行得到可观察结果 y 。

语义 Plan 演算

facade 是纯 Plan 代数。它的构造子携带足够信息,使下层可以解释并行工作,而无需跨越函数值 或运行时状态边界。

```
ScalarKind ::= Int
              | UInt
              | Int64
              | UInt64
              | Float
              | Double
```

图 1 标量家族。

```
Schedule ::= Static
            | Dynamic
            | Guided
```

图 2 调度类别。

```
RuntimeConfig ::= {
    threads : Nat ,
    chunk : Nat ,
    schedule : Schedule
}
```

图 3 运行时配置记录。

```
AccessMode ::= ReadOnly
              | WriteOnly
              | ReadWrite
```

图 4 边界访问模式。

BufferRef ::= (BufId , ScalarKind , Len , AccessMode , LayoutWitness)

图 5 facade 层 buffer 引用。

LawClass ::= StrictAssoc
| DeterministicTree

图 6 规约律类别。

MonoidLaw ::= (LawClass , OpId , IdentityId , AssocWitness)

图 7 幺半群描述子。

Plan ::= MapPlan (RuntimeConfig , OpId , BufferRef , BufferRef)
| ReducePlan (RuntimeConfig , MonoidLaw , BufferRef , OutRef)
| ScanPlan (RuntimeConfig , MonoidLaw , BufferRef , BufferRef)
| MapReducePlan (RuntimeConfig , OpId , MonoidLaw , BufferRef , OutRef)

图 8 并行 Plan ADT。

version 1 禁止任意 MoonBit 函数值与任意 JavaScript 宿主函数进入 Plan。

定义 (Plan 良构性) . 记 $\text{PlanWF}(p)$ 当且仅当:

- `threads > 0`;
- 只要请求 chunk 化并行执行, 就必须满足 `chunk > 0`;
- 空输入 `map` 与 `scan` 只有在其 shape 义务仍能唯一确定空结果 buffer 时才可接受;
- 空输入 `reduce` 只有在 `IdentityId` 能解析到该 `MonoidLaw` 的受审计 identity 元素时才可接受, 否则必须在 lowering 前拒绝;
- 每个 buffer 引用都携带显式 scalar kind、长度、access mode 和 layout witness;
- 每个 operator id 都能解析到一个受信任的已注册 kernel descriptor;
- 每个 monoid law 描述子都携带显式 law class、operator id、identity id, 以及 associativity 或 deterministic-tree witness id;
- 输入与输出长度满足构造子特定的 shape 义务。

定义 (规约律可接受性) . 记 $\text{LawOK}(\mu, \tau)$ 当且仅当:

- 若 $\text{ClassOf}(\mu) = \text{StrictAssoc}$, 则 $\text{AssocWitness}(\mu)$ 表示元素类型 τ 上的严格结合律; 且
- 若 $\text{ClassOf}(\mu) = \text{DeterministicTree}$, 则 $\tau \in \{\text{Float}, \text{Double}\}$, 并且 $\text{AssocWitness}(\mu)$ 表示与规范化 descriptor 绑定的固定规约树 witness。

浮点 `reduce` 与 `scan` 只能通过 `DeterministicTree` 被准入; 本规格不把 IEEE 754 加法或乘法视为无约束的代数幺半群。

定义 (浮点符合性 profile) . 记 $\text{FPPProfile}(t)$ 为目标 t 声明的目标局部浮点环境, 包括 rounding mode、contraction policy、NaN 与 infinity 处理、subnormal 处理, 以及解释 `DeterministicTree` 结果所需的其他条件。

ABI 布局同构

只有在 MoonBit 布局与 C 布局同构时, 才允许 zero-copy 借用或表示保持不变的 transfer。

定义（边界视图布局）. 对活动 backend 目标上的 layout-bearing buffer view, 定义边界视图记录:

```
lv_view ::= {
    addr : UIntPtr ,
    len : UInt64 ,
    kind : UInt32 ,
    mode : UInt32 ,
    stride : UInt64
}
```

图 9 规范边界 view。

规范 boundary ABI 不是从未说明的机器字宽 w 推导出来的; 它必须由一个受审计的目标 profile $\text{ABIPProfile}(t)$ 选定。

定义（边界 ABI profile）. $\text{ABIPProfile}(t)$ 必须声明目标 t 上 boundary record 所使用的唯一宽度、对齐与 endian 合同, 并给出 UIntPtr 、 UInt64 与 UInt32 的映射。

并要求在 $\text{ABIPProfile}(t)$ 下满足下列布局方程:

$$\begin{aligned} \text{sizeof}(\text{lv_view}) &= \text{LVSize}(\text{ABIPProfile}(t)) \wedge \text{alignof}(\text{lv_view}) = \text{LVAlign}(\text{ABIPProfile}(t)) \\ \text{offset}(\text{addr}) &= 0 \wedge \text{offset}(\text{len}) = \text{OffLen}(\text{ABIPProfile}(t)) \wedge \text{offset}(\text{kind}) = \text{OffKind}(\text{ABIPProfile}(t)) \\ \text{offset}(\text{mode}) &= \text{OffMode}(\text{ABIPProfile}(t)) \wedge \text{offset}(\text{stride}) = \text{OffStride}(\text{ABIPProfile}(t)) \end{aligned}$$

定义（布局记录）. 对任意标量或 buffer 元素类型 τ , 定义:

$$\text{Layout}(\tau) = (\text{size}(\tau), \text{align}(\tau), \text{stride}(\tau), \text{endian}(\tau))$$

定义（布局同构）. 记 $\text{Iso}(\tau_M, \tau_C)$ 当且仅当:

$$\text{Layout}(\tau_M) = \text{Layout}(\tau_C)$$

且两边在标量解释、元素宽度和连续遍历语义上完全一致。

约束（边界布局 Gate）.

$$\text{Iso}_{\text{LayoutOK}}^{\tau_M, \tau_C}(\tau_M, \tau_C)$$

若 LayoutOK 失败, 则实现只能显式拷贝到一个符合布局的表示, 或直接拒绝请求。不得静默重解释布局。

LayoutOK 还要求活动目标为承载该 view 的路径声明一个受审计的 $\text{ABIPProfile}(t)$ 。实现不得依据偶然的宿主 ABI 兼容性替换记录布局。

线性所有权状态机

边界合同被建模为一个关于所有权资源上下文的有限状态转移系统。

所有权状态。对任意 buffer 身份 b ，允许的状态只有 $\text{Own_M}(b)$ 、 $\text{Borrow_MR}(b)$ 、 $\text{Borrow_C}(b)$ 、 $\text{Own_C}(b)$ 和 $\text{Returned_M}(b)$ 。

$\text{Returned_M}(b)$ 是一个可观察的“返还完成”中间态：C 可见 authority 已经放弃，但在 rehydration 成功前，MoonBit 的可变 authority 尚未恢复。

转移规则。

$$\begin{array}{c}
\frac{\Gamma \vdash \text{Own_M}(b) \wedge \text{LayoutOK}(\tau_M, \tau_C) \wedge \text{Live}(b) \wedge \text{ReadOnly}(q)}{\Gamma \rightarrow (\Gamma - \{\text{Own_M}(b)\}) \cup \{\text{Borrow_MR}(b), \text{Borrow_C}(b)\}} \quad (\text{Borrow-Step}) \\
\frac{\Gamma \vdash \text{Own_M}(b) \wedge \text{LayoutOK}(\tau_M, \tau_C) \wedge \text{Exclusive}(b) \wedge \text{TransferOK}(q)}{\Gamma \rightarrow (\Gamma - \{\text{Own_M}(b)\}) \cup \{\text{Own_C}(b)\}} \quad (\text{Transfer-Step}) \\
\frac{\Gamma \vdash \text{Own_C}(b) \wedge \text{Received}(b) \wedge \text{Rebind}(b)}{\Gamma \rightarrow (\Gamma - \{\text{Own_C}(b)\}) \cup \{\text{Returned_M}(b)\}} \quad (\text{Return-Step}) \\
\frac{\Gamma \vdash \text{Returned_M}(b) \wedge \text{RehydrateOK}(b)}{\Gamma \rightarrow (\Gamma - \{\text{Returned_M}(b)\}) \cup \{\text{Own_M}(b)\}} \quad (\text{Rehydrate-Step}) \\
\frac{\Gamma \vdash \text{Borrow_MR}(b) \wedge \text{Borrow_C}(b) \wedge \text{BorrowDone}(b)}{\Gamma \rightarrow (\Gamma - \{\text{Borrow_MR}(b), \text{Borrow_C}(b)\}) \cup \{\text{Own_M}(b)\}} \quad (\text{Discharge-Step})
\end{array}$$

图 10 线性所有权转移。

线性条件。任何可达上下文都不能同时包含同一 b 的 $\text{Own_M}(b)$ 与 $\text{Own_C}(b)$ ；任何活动中的 $\text{Borrow_C}(b)$ 都会阻止 MoonBit 侧突变和 free，直到 borrow token 通过 **Discharge-Step** 被释放。

边界承载 Payload

FFI 边界传输的是 Plan 与布局检查后的 view，而不是任意对象图。

Payload 骨架。

```
typedef struct {
    PlanTag plan_tag;
    CfgBits cfg_bits;
    ViewVec views;
    OpBits op_bits;
    LawBits law_bits;
} Payload;
```

每个 view 项都携带：

- buffer 身份或边界 view handle；
- scalar kind 标签；
- 长度；
- access mode；
- layout witness 或 layout class 标签；
- ownership mode (**Borrow** 或 **Transfer**)。

边界可接受性。Adm(t , q) 仅当以下条件都成立：

- payload 标签属于目标 t 的 payload 语法；
- 每个承载 view 都满足 LayoutOK；
- q 请求的每个 ownership transition 都在线性状态机中合法；

- 每个 operator 或 monoid id 都能在目标 t 上解析到一个被准入的受信任 descriptor 或 law。

包装层解析与描述子形成

C wrapper 或 JS/N-API 边界是从 payload 语法到 runtime descriptor 的解释器。

Runtime descriptor。

```
typedef struct {
    PlanTag plan_tag;
    CfgNorm cfg_norm;
    ChunkPolicy chunk_policy;
    BufSliceVec buf_slices;
    KernelDesc kernel_desc;
    LawHandle law_handle;
} Descriptor;
```

Kernel descriptor 与 registry。

```
typedef struct {
    KernelId kernel_id;
    FnPtr fn_ptr;
    PurityWitness purity_witness;
    FootprintWitness footprint_witness;
    LayoutContract layout_contract;
    ArityMeta arity_meta;
    LawMeta law_meta;
    TargetMeta target_meta;
} KernelDesc;
```

$\text{KernelRegistry}(t)$ 是目标 t 上从 KernelId 到 KernelDesc 的静态受审计映射。 KernelId 是 plan payload 中唯一允许出现的 kernel 级标识符； FnPtr 仅属于实现层，绝不跨越 MoonBit plan 边界。

解析关系。

记 $\text{ParseCore}(t, q, p, c, V, k, \mu)$ 当且仅当：

- $\text{DecodePlan}(q) = p$;
- $\text{NormCfg}(q) = c$;
- $\text{NormViews}(q) = V$;
- $\text{ResolveKernel}(\text{KernelRegistry}(t), q) = k$ ；以及
- $\text{ResolveLaw}(q) = \mu$ 。

记 $\text{ViewsOK}(V)$ 当且仅当每个 $v \in V$ 都同时满足 $\text{LayoutOK}(v)$ 与 $\text{OwnerOK}(v)$ 。

记 $\text{ParseReady}(t, p, V, k, \mu)$ 当且仅当 $\text{KernelOK}(k, p, V, \mu, t)$ 。

$$\frac{\text{ParseCore}(t, q, p, c, V, k, \mu) \wedge \text{ViewsOK}(V) \wedge \text{ParseReady}(t, p, V, k, \mu)}{\text{parse}(t, q) \mapsto \text{Descriptor}(p, c, \text{ChunkPolicy}(c), V, k, \mu)}$$

图 11 包装层解析关系。

描述子保真规则。产出的 descriptor 必须保持 plan 的构造子类别、每个 buffer 的 scalar kind 与 layout class、每个边界 view 所附带的 ownership mode、plan 所选择的 KernelId 与其从受信任 registry 中解析出的审计 descriptor，以及 plan 构造子要求的 shape 义务。

Fork-Join 操作语义

我们给出一个关于配置 $\text{Conf}(\mathbf{C}, \mathbf{H}, \mathbf{S})$ 的标记小步语义，其中 $\mathbf{H}$ 是宿主堆， $\mathbf{S}$ 是 worker 本地 store。第三部分会把这台抽象机器细化为适用于物理 C/OpenMP 实现的显式 fail-closed 错误发布合同。

Worker 语法。

```
Worker ::= Read ( Addr )
        | Write ( Addr , Val )
        | MapStep ( Op , Addr , Addr )
        | Seq ( Worker , Worker )
        | Done ( Val )
```

图 12 worker 语法。

Read 规则。

$$\frac{H(a) = v}{\text{Conf}(\text{Read}(a), H, S) \rightarrow \text{Conf}(\text{Done}(v), H, S)}$$

图 13 读步骤。

Write 规则。

$$\frac{\text{Writable}(a, S)}{\text{Conf}(\text{Write}(a, v), H, S) \rightarrow \text{Conf}(\text{Done}(v), H[a := v], S)}$$

图 14 写步骤。

Map 规则。

$$\frac{H(a_s) = x \wedge \text{Writable}(a_d, S) \wedge \text{Apply}(\text{op}, x) = y}{\text{Conf}(\text{MapStep}(\text{op}, a_s, a_d), H, S) \rightarrow \text{Conf}(\text{Done}(y), H[a_d := y], S)}$$

代码 1 映射步骤。

Sequence 规则。

$$\frac{\text{Conf}(C_1, H, S) \rightarrow \text{Conf}(C_1', H', S')}{\text{Conf}(\text{Seq}(C_1, C_2), H, S) \rightarrow \text{Conf}(\text{Seq}(C_1', C_2), H', S')}$$

$$\frac{\text{Conf}(C_2, H, S) \rightarrow \text{Conf}(C_2', H', S')}{\text{Conf}(\text{Seq}(\text{Done}(v), C_2), H, S) \rightarrow \text{Conf}(C_2', H', S')}$$

图 15 顺序组合步骤。

Footprint 合同。 $\mathbf{R}(\mathbf{C})$ 是 $\mathbf{C}$ 单步读取的地址集合， $\mathbf{W}(\mathbf{C})$ 是 $\mathbf{C}$ 单步写入的地址集合， $\mathbf{FP}(\mathbf{C}) = \mathbf{R}(\mathbf{C}) \cup \mathbf{W}(\mathbf{C})$ 。

Descriptor 元数据区域。对每个解析后的 descriptor $\mathbf{d}$ ，定义 $\mathbf{MetaOf}(\mathbf{d})$ 为只读元数据区域，其中存放规范化的 runtime configuration、operator handle 及其参数、law handle，以及 spawned workers 所需的每计划常量。

Chunk 分块合同。 $\text{Chunks}(n, k) = [I_0, \dots, I_{(k-1)}]$ 只有在区间覆盖 $[0, n)$ 且两两不相交时才成立。

Worker 实例化合同。若 worker C_i 由切片 I_i 实例化，则 $\text{FP}(C_i)$ 必须落在 $\text{AddrOf}(I_i)$ union $\text{MetaOf}(d)$ 之内，且 $\text{MetaOf}(d)$ 必须保持只读。

Fork 与 Join。

$$\frac{\frac{\text{Chunks}(n, k) = [I_0, \dots, I_{k-1}] \wedge \forall i. \text{Spawn}(d, I_i) = C_i}{\text{Conf}(\text{Fork}(d), H, S) \rightarrow \text{Conf}(\text{Par}([C_0, \dots, C_{k-1}]), H, S)} \quad \forall i. C_i = \text{Done}(y_i)}{\text{Conf}(\text{Par}([C_0, \dots, C_{k-1}]), H, S) \rightarrow \text{Conf}(\text{Done}(\text{Fold}(\mu, \text{TreeOf}(d), [y_0, \dots, y_{k-1}])), H, S)}$$

图 16 fork 与 join 步骤。

浮点组合规则。若 $\text{ClassOf}(\mu) = \text{DeterministicTree}$ ，则 $\text{TreeOf}(d)$ 属于 descriptor normalization 的一部分，并唯一决定 split-combine 顺序。worker 调度上的重排不会改变由 d 选定的组合树。该规则保证在单一目标局部 $\text{FPPProfile}(t)$ 下重复执行的结果稳定；跨目标相等只在被比较目标声明了兼容的受审计浮点 profile 时才被要求。

并行侧条件。若 $\text{Conf}(C_i, H, S) \rightarrow \text{Conf}(C_i', H', S')$ ，则提升到 $\text{Par}(\dots)$ 的步骤只在 $\text{W}(C_i) \cap \text{FP}(C_j) = \text{emptyset}$ 对所有 $j \neq i$ 都成立时才可接受。

JS ArrayBuffer 边界合同

JavaScript 边界被建模为宿主视图上的目标特化转移系统。

JS 视图语法。

$$\text{JSView} ::= (\text{ArrayBufferKind}, \text{Offset}, \text{Len}, \text{ScalarKind}, \text{AccessMode}, \text{Share})$$

图 17 JavaScript 边界视图。

边界规则。

$$\frac{\text{Kind}(v) = \text{SharedArrayBuffer} \wedge \text{Access}(v) = \text{ReadOnly} \wedge \text{LayoutOK}(v) \wedge \text{AtomicOK}(v)}{(\text{Own_M}(b), v) \rightarrow (\text{Borrow_MR}(b) * \text{Borrow_C}(b), v)} \\ \frac{\text{Kind}(v) = \text{ArrayBuffer} \wedge \text{CopyBytes}(b) = b' \wedge \text{CopyOwner}(b') = \text{HostRuntime}}{(\text{Own_M}(b), v) \rightarrow (\text{Own_M}(b) * \text{Own_C}(b'), v)} \\ \frac{\text{TransferOK}(v) \wedge \text{Tombstone}(v)}{(\text{Own_M}(b), v) \rightarrow (\text{Own_C}(b), v)}$$

图 18 JS 边界上的 borrow、copy 与 transfer。

锁定侧条件。共享借用的每个推导都携带宿主侧锁义务：若 **Share** 中不包含相应原子纪律，则共享区域上的写入不可接受。

JS 生命周期义务。对 copy 规则，宿主 runtime 持有 b' 并且必须恰好释放一次，可通过显式 release 或 finalizer 完成。对 transfer 规则，宿主对象在 transfer 准入后立即进入 tombstone 状态；explicit return 优先于 finalizer 清理，而 finalizer 清理优先于 fault 情况下的泄漏式处理。若 transfer 之后 runtime 准入或 worker 执行 fail closed，则目标必须为仍由 C 可见侧持有的 buffer 发布一条确定性的清理路径。

分离与数据竞争检查

空间分离谓词。

$$\text{SepChunks}(b, [I_0, \dots, I_{k-1}]) = \forall i \neq j. \text{AddrOf}(I_i) \cap \text{AddrOf}(I_j) = \emptyset$$

并行可接受性。

$$\text{ParOK}([C_0, \dots, C_{k-1}]) = \forall i \neq j. W(C_i) \cap \text{FP}(C_j) = \emptyset$$

定理（数据竞争自由）。若

1. $\text{KernelSafe}(d)$ 成立，即已解析的受审计 kernel descriptor 满足 PurityWitness 、 FootprintWitness 、 LayoutContract 与 LawMeta ；
2. $\text{SepChunks}(b, [I_0, \dots, I_{(k-1)}])$ ，
3. 每个 worker C_i 都由 I_i 生成，
4. $\text{FP}(C_i) \text{ subset.eq } \text{AddrOf}(I_i) \text{ union } \text{MetaOf}(d)$ 对所有 i 成立，且
5. $\text{MetaOf}(d)$ 是只读的，

则每个可达配置 $\text{Conf}(\text{Par}([C_0, \dots, C_{(k-1)}]), H, S)$ 都满足 $\text{ParOK}([C_0, \dots, C_{(k-1)}])$ 。

证明。对推导长度归纳。基例由 chunk-spawn 不变式和 SepChunks 的定义直接得到。前提 (1) 使该定理在“受审计 kernel 可接受性”假设下闭合，从而 worker footprint 的可推导性仅依赖 plan 数据。归纳步中，每次只有一个 worker C_i 进行小步。由前提 (4)， C_i 的写入落在 $\text{AddrOf}(I_i)$ 中；对于任意 $j \neq i$ ，前提 (4) 给出 $\text{FP}(C_j) \text{ subset.eq } \text{AddrOf}(I_j) \text{ union } \text{MetaOf}(d)$ 。前提 (2) 保证 $\text{AddrOf}(I_i)$ 与 $\text{AddrOf}(I_j)$ 不相交，前提 (5) 禁止写入 $\text{MetaOf}(d)$ 。因此 $W(C_i) \text{ inter } \text{FP}(C_j) = \text{emptyset}$ 对所有 $j \neq i$ 仍成立，故并行配置保持 ParOK 。

推论（所有权安全）。若 $\text{Borrow}_C(b)$ 活动， MoonBit 不能导出释放或突变 b 的步骤；若 $\text{Own}_M(b) \rightarrow \text{Own}_C(b)$ 已经发生，则 MoonBit 不能在没有显式 Return-Step 与后续 Rehydrate-Step 的情况下恢复对 b 的可变 authority；若 $\text{Borrow}_{MR}(b) * \text{Borrow}_C(b)$ 活动，则 $\text{Own}_M(b)$ 只能通过 Discharge-Step 恢复。

第二部分：实现化与符合性

Native 实现化

native 实现化路径为：

$\text{MoonBit facade} \rightarrow \text{MoonBit C FFI} \rightarrow \text{C wrapper} \rightarrow \text{runtime descriptor} \rightarrow \text{C runtime}$

一个符合性的 native 实现必须：

- 只下放第一部分允许的 Plan ADT；
- 对每条被准入的边界路径保持一个受审计的 $\text{ABIProfile}(t)$ ，或在布局不同时显式执行拷贝；
- 按线性所有权状态机实现 borrow 与 transfer 边；
- 只在满足第一分离合同的前提下调度可写 chunk。

JavaScript 实现化

JavaScript 实现化路径为:

MoonBit facade -> MoonBit JS FFI -> JS wrapper -> N-API addon -> runtime descriptor -> C runtime

一个符合性的 JavaScript 实现必须:

- 把 typed-array 兼容视图下放为携带显式元素种类标签的 payload view;
- 拒绝所有不满足布局与所有权义务的宿主对象;
- 与 native 实现保持相同的 Plan 指称和 split-combine 合同;只有当双方声明兼容的 FPProfile 时,才要求更强的跨目标相等性;
- 在 runtime 执行开始前,为 copy 与 transfer 路径定义 owner、tombstone、finalizer 与 explicit-return 行为;
- 在 runtime 执行开始之前完成 borrow 或 transfer 纪律检查。

Facade 义务

一个符合性的 facade 必须:

- 只暴露纯 Plan 构造子和显式执行入口;
- 拒绝把任意函数值环境或任意宿主函数编码进 Plan;
- 为每个导出的边界 view 绑定显式 ownership mode 与 access mode;
- 暴露与真实目标支持在外延上完全一致的 capability 查询。

Runtime 义务

一个符合性的 runtime 必须:

- 只通过第一部分定义的构造子类别解释 descriptor;
- 对所有不满足分离或 side condition 的 descriptor 进行执行前拒绝;
- 保持每个承载 view 所附带的 ownership mode;
- 将失败提升为显式状态类别,而不是返回部分成功;
- 禁止异常逃逸、panic 传播或栈展开跨越 FFI 边界;
- 在 worker 执行开始前,对非同构视图执行确定性的布局降级或拒绝。

不支持表面

以下内容不属于 version-1 合同:

- 任意 MoonBit 函数值在 foreign worker 线程上的执行;
- 任意 JavaScript 宿主函数在 native worker 执行期间的运行;
- 对非同构 ABI 记录或数组进行隐式布局重解释;
- 不经过边界状态转移的隐式所有权变更;
- 无法通过分离与 split-combine 约束验证的 chunk 调度。

符合性

一个实现只有在满足以下条件时才可以声称符合本规格:

- 它的 MoonBit-facing API 只构造第一部分允许的 Plan ADT;
- 它的边界 payload 保持显式 layout witness 与 ownership witness;
- 它的 wrapper 满足解析保真与所有权保真;
- 它的 runtime 满足 chunk 正交性和 split-combine 健全性;
- 所有非法、非同构或生命周期不安全请求都必须在执行前 fail closed。

符合性传递

忠实实现。只有当实现满足以下条件时，才可称为忠实的：

- 每个被接受 payload 都是某个良构 Plan 的 lowering;
- 每个 borrow / transfer 边都遵守线性所有权状态机;
- 每个可写 chunk 调度都满足 SepChunks;
- 每个 reduction / scan combine 路径都由有效 monoid 描述子支撑。

符合性规则。如果忠实实现接受某个 plan p ，则执行 p 的可观察结果和所有权轨迹必须留在本规格范围内。如果它拒绝 p ，则该拒绝必须是本规格允许的 fail-closed 结果。对浮点 reduce 与 scan，“可观察结果”的含义是：在单一已准入目标 profile 下重复执行结果稳定；跨目标相等只在显式兼容的受审计浮点 profile 之间被要求。

验证场景

1. 提交一个源视图与目标视图布局不同构的 Plan。预期结果：显式拷贝到符合布局的表示，或直接 fail-closed 拒绝。
2. 提交一个只读 borrow Plan，并在 borrow 释放前尝试从 MoonBit 侧突变该 buffer。预期结果：该 mutation 被拒绝或被 borrow 纪律阻塞。
3. 提交一个包含重叠区间的可写 chunk 调度。预期结果：descriptor 形成或 runtime 准入在执行前失败。
4. 提交一个使用无有效 monoid 描述子的 reduce 或 scan Plan。预期结果：该 Plan 在 fork-join 执行前被拒绝。
5. 提交一个 transfer Plan，并尝试通过旧 alias 在 MoonBit 侧释放该 buffer。预期结果：可变别名权限不存在，因此该操作被拒绝。
6. 在 native 与 JavaScript 两个目标上执行等价的浮点 reduce Plan，但两边的浮点 profile 不兼容。预期结果：每个目标内部都保持确定性，但不声称跨目标相等。
7. 在声明兼容浮点 profile 的目标上执行等价的浮点 reduce Plan。预期结果：两边保持相同的已声明可观察结果和所有权纪律。
8. 提交一个 transfer-return-reuse Plan，并在 Rehydrate-Step 之前尝试 reuse。预期结果：在 rehydration 成功之前，reuse 被拒绝。
9. 提交一个 JS copy 或 transfer Plan，并分别触发 explicit return、finalizer cleanup 和 fail-closed runtime admission。预期结果：文档声明的生命周期优先级与单次释放义务成立。

Closure Checklist

只有在以下条件全部成立时，本规格才算闭合：

1. 每个导出执行请求都能由 Plan ADT 表示。
2. 每个边界 view 都携带显式 layout witness 与 ownership witness。
3. 每条 zero-copy 路径都满足布局同构。

4. 每条 borrow 路径都冻结 MoonBit 侧突变直到 borrow 释放。
5. 每条 transfer 路径都消费 MoonBit 侧的可变 authority，并且只通过显式 return 加 rehydration 恢复。
6. 每个可写 chunk 调度都经过分离检查验证。
7. 每条 reduction 或 scan combine 路径都由有效 monoid law 支撑。
8. 不存在任何异常、panic 或 abort 路径跨越 FFI 边界逃逸。
9. 每条 zero-copy 准入路径都完成字节级布局与对齐检查。
10. 每条 JS copy 与 transfer 路径都具有一个闭合的清理故事，包括 finalizer 与 fail-closed 情况。
11. 每个 witness 义务都映射到一个可审计的 artifact 格式。
12. 每个非法输入向量都通过显式状态失败，而不是崩溃或触发未定义行为。
13. 文档中引用的每项保证都已显式陈述。

第三部分：实现一致性验证

异常控制流与 Fail-Closed 语义

第一部分把 worker 执行建模为成功的局部小步。忠实的物理实现还必须闭合 MoonBit、C wrapper 与 native runtime 中可能出现的所有异常控制流路径。

状态分类。每个实现可见的失败都必须属于一个封闭的状态家族：

```
RejectReason ::= ParseReject
               | LayoutReject
               | OwnershipReject
               | DescriptorReject
               | OperatorReject
               | KernelLookupReject
               | KernelWitnessReject
               | KernelLayoutReject
               | KernelArityReject
               | KernelLawReject
               | BorrowLifecycleReject
               | DeterministicTreeReject
               | TargetSupportReject
```

图 19 封闭的拒绝原因家族。

```
Status ::= Ok
         | Reject ( RejectReason )
         | Fault ( RuntimeFault )
         | Worker ( WorkerFault )
```

图 20 带标签的符合性状态代数。

边界结果承载。每个供运行时执行使用的 MoonBit 暴露 native kernel 与 wrapper 入口，都必须返回一个 ABI-safe 的结果封装：

```
typedef struct {
    Status status;
    Payload payload;
} FFIResult_Payload;
```

或者返回目标特化但在可观察行为上等价的记录。任何栈展开、panic 传播或宿主异常传播都不得跨越 FFI 边界。

阶段映射义务。每个非 `Ok` 状态都必须对应以下符合性阶段之一：

- facade 构造拒绝；
- wrapper parse 拒绝；
- descriptor 准入拒绝；
- 并行域前的 runtime 拒绝；或
- 并行域内 worker 的 fail-closed 发布。

该阶段映射属于可观察合同的一部分，MoonBit 侧必须能够通过模式匹配，或通过可观察行为等价的 目标 API 恢复这一信息。

共享 worker 状态。在 native 并行域内，worker 只能通过一个共享的原子状态字通信失败，该状态字初始为 `Ok`。

```
typedef struct {
    _Atomic uint32_t code;
} AtomicStatusWord;
```

Worker 错误发布规则。若某 worker 检测到错误状态 `e != Ok`，则它必须：

1. 以 release 语义把 `e` 写入 `AtomicStatusWord`，除非先前已经可见某个非 `Ok` 状态；
2. 除线程本地清理外，不再继续发出新的写入；
3. 只执行清理或 early-exit 步骤，直到并行域 join。

其余 worker 必须在准入点或循环检查点以 acquire 语义观察共享状态字；一旦观察到非 `Ok` 值，就必须降级为确定性的 early exit。

CFG 闭包义务。对于 `#pragma omp parallel` 内每一条可达控制流路径，实现必须通过静态分析、代码审计或等价论证证明：它只能以下列方式之一终止：

- 以 `Ok` 正常完成 worker；
- 显式发布一个非 `Ok` 状态并完成本地清理；
- 因观察到先前发布的非 `Ok` 状态而 early exit。

不得存在任何通过 `abort()`、未捕获异常逃逸、跨边界栈展开或未定义控制转移终止的路径。

布局同构的实现化与安全降级

第一部分把 `LayoutOK` 抽象定义为公理式条件。符合性的实现必须通过字节级静态断言与动态准入检查来将它实现化。

C 边界记录义务。对 `lv_view`，C 侧实现必须通过属性、packing 指令或等价的 ABI 稳定机制，保持第一部分给出的 规范 size、alignment 与 field offset 方程。

这些检查必须绑定到一个已声明的 `ABIProfile(t)`，而不是绑定到推断出来的宿主字宽。

编译期布局出清。native wrapper 必须在编译期出清下列条件：

- `sizeof(lv_view)` 与 `alignof(lv_view)`;
- `addr`、`len`、`kind`、`mode` 与 `stride` 的字段偏移;
- 活动目标 profile 下 `UIntPtr`、`UInt64` 与 `UInt32` 的宽度检查;
- 活动目标合同所要求的标量宽度与 `endian` 假设。

这些检查必须由 `static_assert` 或可观察行为等价的编译期机制表达。

动态 zero-copy gate。在某个边界 view 以无拷贝方式被准入为 borrow 或 transfer 之前，descriptor 形成必须检查：

- base pointer 对所需元素对齐的模运算结果为零;
- stride 等于该标量种类声明的连续遍历规则;
- 总字节跨度有界且不发生算术溢出;
- 源与目标的 layout witness 在标量解释和宽度上一致。

若任一检查失败，实现不得原地重解释该 view。

确定性降级规则。对每个目标，实现必须为非同构视图声明下列结果之一：

- `CopyConforming`：复制字节到符合布局的表示后继续；或
- `RejectNonIsomorphic`：在 runtime 准入前直接拒绝。

该选择必须对给定目标路径稳定成立，不得依赖偶然的宿主 ABI 兼容性。

对于每条被准入的目标路径，实现都必须把 `ABIProfile(t)` 及所选降级规则文档化为一个可审计的边界合同。

算子纯度与 Plan 准入

数据竞争自由定理依赖于 worker footprint 可以仅由 plan 数据推出。因此 facade 与 lowering 边界必须拒绝任何把不可验证 effect 偷运过 FFI 边界的可执行内容。

Kernel registry 表面。version 1 的可执行 Plan 只能携带能在活动目标的静态受审计 `KernelRegistry(t)` 中解析的 `KernelId` 引用。任意 MoonBit 闭包、任意 JavaScript 宿主函数、未入册的 native 函数指针，以及捕获了可变环境的值，都不是可接受的 plan payload。

每个受信任的 `KernelDesc` 都必须携带：

- `KernelId`，即 plan 唯一可见的 kernel 级标识符;
- 隐藏的 native `FnPtr`，其不得跨越 MoonBit plan 边界;
- `PurityWitness`;
- `FootprintWitness`;
- `LayoutContract`;
- `ArityMeta`;
- `LawMeta`; 以及
- `TargetMeta`。

动态的任意 kernel 注册不属于本合同。符合性声明只能依赖静态受审计 registry 中已有的 descriptor。

对浮点 `reduce` 与 `scan`, 额外的准入条件是必须提供 `DeterministicTree` 律 witness, 使 runtime descriptor 为被接受的执行固定唯一的组合树。

安全抽象义务。一个符合性的 MoonBit-facing API 必须确保: 任何通过类型检查并被准入的执行请求, 都会 lowering 为一个其 worker footprint 仅由以下信息决定的 plan:

- plan 构造子;
- 显式 buffer view 与 layout witness;
- 受信任的 `KernelId` 与已解析的 `KernelDesc`;
- 被 `PlanWF` 允许的 runtime configuration 字段。

任何允许隐藏可写别名、逃逸可变引用, 或携带无法依据 `SepChunks`、所有权线性与边界 access mode 证明其安全性的 effectful 算子体的构造路径, 都必须被拒绝。

Witness 义务。对每个被准入的 descriptor `k = KernelDesc(...)`:

- `PurityWitness(k)` 必须证明执行在声明的输入/输出 view 和 `MetaOf(d)` 之外没有副作用;
- `FootprintWitness(k)` 必须证明 worker 写入被限制在其分配到的 chunk 内, 而任何超出 chunk 的读取都仅限于声明的只读输入或 `MetaOf(d)`;
- `LayoutContract(k)` 必须在 runtime 执行前针对所有承载的边界 view 被出清;
- `ArityMeta(k)` 必须与 plan 的构造子类别和 access mode 匹配; 且
- `LawMeta(k)` 必须满足 `mu` 所要求的规约律义务, 包括浮点归约所需的 deterministic-tree 要求。

这些义务若不满足, 执行必须通过 `Status` 中专用的拒绝分支被拒绝。

可审计 witness 格式。每个携带 witness 的 registry 条目都必须暴露一个可独立复审的 artifact bundle。最小可接受 bundle 包含:

- 稳定的 kernel 标识符, 以及与被审计实现绑定的版本号或哈希;
- 对 purity、footprint、layout、arity 与 law 声明的机器可检查摘要或表格化摘要;
- 这些声明成立时所依赖的目标或 `ABIProfile(t)` 与浮点 profile 假设;
- 每项声明所采用的审计方法: 证明 artifact、静态分析结果、测试证书, 或人工审计记录; 以及
- 以第三方可区分“缺失”“拒绝”“接受”证据的形式记录的审计结论。

语义义务本身具有规范性; artifact 格式则是使符合性声明可复审而非自证的最小可接受证据。

基于契约的验证矩阵

符合性声明必须由一组可执行的验证矩阵支撑, 该矩阵覆盖接受、拒绝与 fail-closed 执行结果。

验证维度。

1. 正向实现化: 等价的 `map`、`reduce`、`scan` 与 `map-reduce Plan` 在已准入的 native 与 JavaScript 目标上保持相同的 plan 指称与所有权轨迹。
2. 异常控制流: 注入的算子故障、分配失败、畸形 payload 与 runtime 准入失败都以显式状态码出现, 而不是以进程终止暴露。

3. 布局与字节级安全：错误对齐、不一致 stride、非同构 scalar kind 与溢出的字节跨度计算，都会按照目标规则触发 CopyConforming 或显式拒绝。
4. 所有权与 chunk 安全：重叠可写 chunk、transfer 后的陈旧 alias、活动 borrow 期间的突变，以及 double-borrow 尝试，都必须在不健全执行发生前失败。
5. 算子准入：携带闭包、宿主函数、未知 KernelId，或其他不可验证内容的 Plan，必须在 facade 或 wrapper 准入阶段被拒绝。
6. 浮点确定性规约：对同一个已准入的浮点 reduce 或 scan descriptor，在 worker 调度不同的情况下重复执行，都必须在单一目标 profile 下得到相同的组合树与相同的可观察结果；跨目标相等性只对已声明兼容的受审计浮点 profile 做测试。
7. 借用生命周期：成功 discharge 会恢复 Own_M(b)；而过早 discharge、重复 discharge，或在没有活动借用时尝试 discharge，都必须被拒绝。
8. Transfer-return 生命周期：成功的 Return-Step 加 Rehydrate-Step 会恢复 Own_M(b)；而在 Returned_M(b) 上 reuse、重复 return，或在没有活动 Own_C(b) 时 return，都必须被拒绝。
9. JS 生命周期：copy 出来的 buffer 只有一个 owner 且只有一条释放路径；transfer tombstone 会阻止陈旧宿主复用；finalizer 与 explicit-return 的顺序遵守文档声明的目标规则。
10. 元数据纪律：worker 在其 chunk footprint 之外只能读取 MetaOf(d)；任何可写或畸形的元数据区域都必须在不健全执行发生前被拒绝。
11. Registry 解析：已知 KernelId 会解析为受审计 descriptor；未知 id 必须通过 KernelLookupReject 拒绝。
12. Witness 校验：缺失 purity、footprint、layout、arity、law、profile 或 target-support 证据时，必须通过对应的专用拒绝分支拒绝。
13. 审计 artifact 完整性：验证矩阵使用到的每个 registry 条目都必须标识其证据 bundle 与审计结论。

拒绝阶段合同。每个负向验证用例都必须说明：

- 被违反的前置条件；
- 拒绝或失败发布发生的阶段：facade 构造、wrapper parse、descriptor 准入、并行域前的 runtime 检查，或并行域内 worker 的 fail-closed 发布；
- 预期的带标签 Status 分支，以及适用时的拒绝子原因。

覆盖义务。只有当验证矩阵中的每个非法输入向量都收敛为本规格定义的显式状态，并且没有任何非法向量能够产生段错误、静默数据损坏或未定义行为时，实现才可称为一致性完备。